

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 5

Виконав
студент групи КН-25-1
Гордієнко В. Ю.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

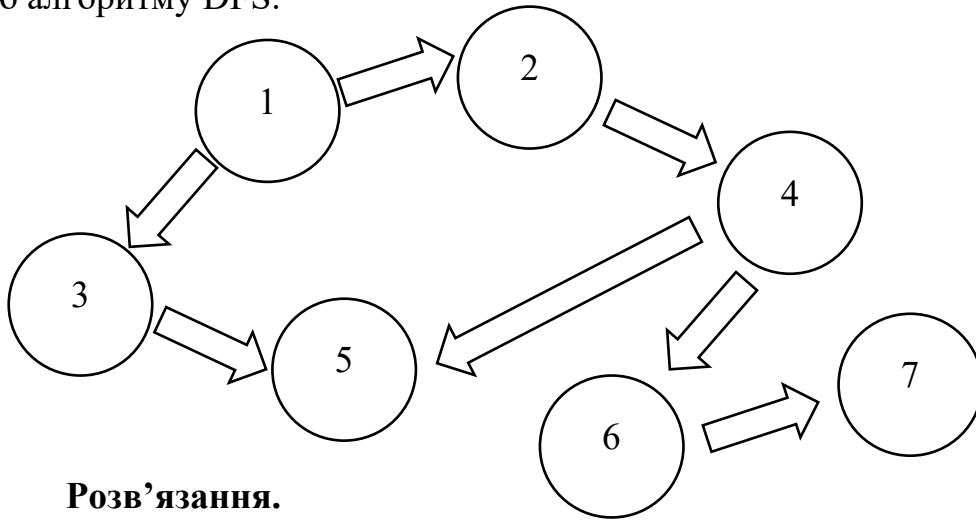
Кременчук 2026

Тема: Графи. Ациклічні графи

Мета: набути практичних навичок розв'язання задач топографічного сортування та оцінювання їх асимптотичної складності.

Порядок виконання роботи

Завдання 6. Задано ациклічний граф: $\{1, 2, 3, 4, 5, 6, 7\}$, $\{(1,2), (1,3), (2,4), (3,5), (4,5), (4,6), (6,7)\}$. Розв'язати задачу топологічного сортування за допомогою алгоритму DFS.



Розв'язання.

1. Початковий стан графа:

Вершини: $\{1, 2, 3, 4, 5, 6, 7\}$

Ребра: $\{(1,2), (1,3), (2,4), (3,5), (4,5), (4,6), (6,7)\}$

2. Початковий стан списку результату L: Порожній.

Для фіксації порядку використовується масив станів вершин (відвідана / невідвідана).

3. Запустимо обхід DFS з першої невідвіданої вершини – вершини 1.

Вершина 1 переходить у стан «в процесі обходу». Переходимо до її першого вихідного сусіда – вершини 2.

4. Запуск DFS для вершини 2.

Вершина 2 переходить у стан «в процесі обходу». Переходимо до її єдиного вихідного сусіда – вершини 4.

5. Запуск DFS для вершини 4.

Вершина 4 переходить у стан «в процесі обходу». Переходимо до її першого вихідного сусіда – вершини 5.

6. Запуск DFS для вершини 5.

У вершини 5 немає вихідних ребер, вона повністю оброблена. Додаємо її до с

писку результату L.

Поточний стан списку $L = [5]$.

7. Повертаємось до вершини 4. Переходимо до її наступного вихідного сусіда – вершини 6. Запуск DFS для вершини 6.

8. Запуск DFS для вершини 6. Переходимо до її єдиного вихідного сусіда – вершини 7.

9. Запуск DFS для вершини 7.

У вершини 7 немає вихідних ребер, вона повністю оброблена. Додаємо її до списку результату L зліва.

Поточний стан списку $L = [7, 5]$.

10. Повертаємось до вершини 6. Усі її вихідні шляхи досліджено, вона повністю оброблена. Додаємо її до списку результату L.

Поточний стан списку $L = [6, 7, 5]$.

11. Повертаємось до вершини 4. Усі її вихідні шляхи досліджено, вона повністю оброблена. Додаємо її до списку результату L.

Поточний стан списку $L = [4, 6, 7, 5]$.

12. Повертаємось до вершини 2. Усі її вихідні шляхи досліджено, вона повністю оброблена. Додаємо її до списку результату L.

Поточний стан списку $L = [2, 4, 6, 7, 5]$.

13. Повертаємось до вершини 1. Переходимо до її другого вихідного сусіда – вершини 3. Запуск DFS для вершини 3.

14. Запуск DFS для вершини 3. Її єдиний сусід (вершина 5) вже повністю оброблений. У вершини 3 більше немає невідвіданих сусідів, вона повністю оброблена. Додаємо її до списку результату L.

Поточний стан списку $L = [3, 2, 4, 6, 7, 5]$.

15. Повертаємось до вершини 1. Усі її вихідні шляхи повністю досліджено, вона оброблена остання. Додаємо її до списку результату L.

Поточний стан списку $L = [1, 3, 2, 4, 6, 7, 5]$.

16. Оскільки всі вершини графа вже успішно відвідані та оброблені, алгоритм завершує свою роботу.

17. **Отже, результуюче топологічне сортування: $L = [1, 3, 2, 4, 6, 7, 5]$**

Висновки: на цій практичній роботі я набув практичних навичок розв'язання задач топографічного сортування та оцінювання їх асимптотичної складності.

Контрольні питання

1. Які переваги і недоліки алгоритму Кана порівняно з алгоритмом DFS для топологічного сортування графа?

Алгоритм Кана (на основі вхідних степеней) ітеративний, не переповнює стек і простіше виявляє цикли. DFS (обхід у глибину) простіший у реалізації, але використовує рекурсію, що небезпечно для глибоких графів.

2. Яка складність часу і пам'яті для кожного з алгоритмів у найгіршому і найкращому випадках?

Для обох алгоритмів часова складність завжди становить $O(V + E)$, а просторова – $O(V)$ у будь-яких випадках, де (V) – вершини, а (E) – ребра.

3. Чи можна застосовувати алгоритм Кана до графів з вагами на ребрах? Як це порівняти з DFS?

Обидва алгоритми можна застосовувати, оскільки ваги ребер взагалі не впливають на топологічний порядок (важливий лише напрямок зв'язків).

4. Як впливає структура графа на швидкість роботи кожного з цих алгоритмів?

Швидкість прямо залежить від кількості ребер. При цьому ланцюгова структура сильніше навантажує стек у DFS, а широка – збільшує розмір черги в алгоритмі Кана.

5. Чи є обмеження використання кожного алгоритму для певних типів графів або завдань?

Обидва алгоритми працюють виключно з орієнтованими ациклічними графами (DAG); за наявності хоча б одного циклу топологічне сортування неможливе.

6. Які варіанти оптимізації можна застосувати для кожного алгоритму з метою поліпшення його продуктивності?

Використання списків суміжності замість матриць, заміна рекурсії на ітеративний стек у DFS та застосування ефективних масивів для підрахунку степеней в алгоритмі Кана.